

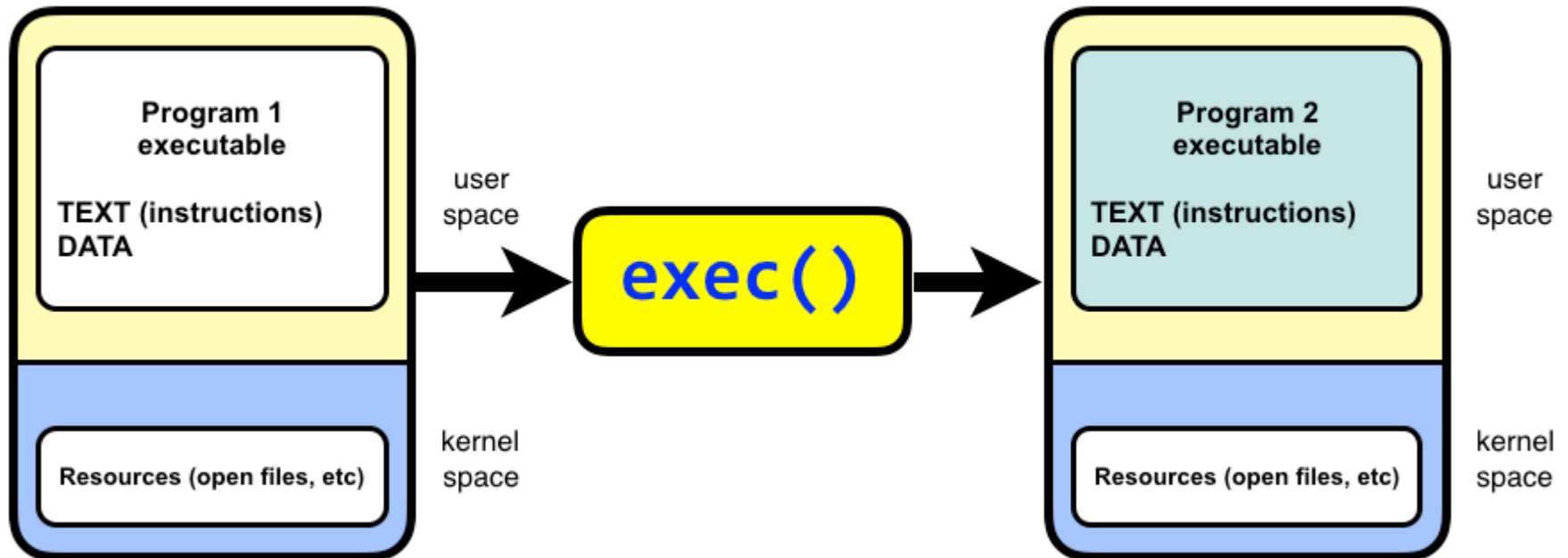


OS – LAB

Process Management – Part II

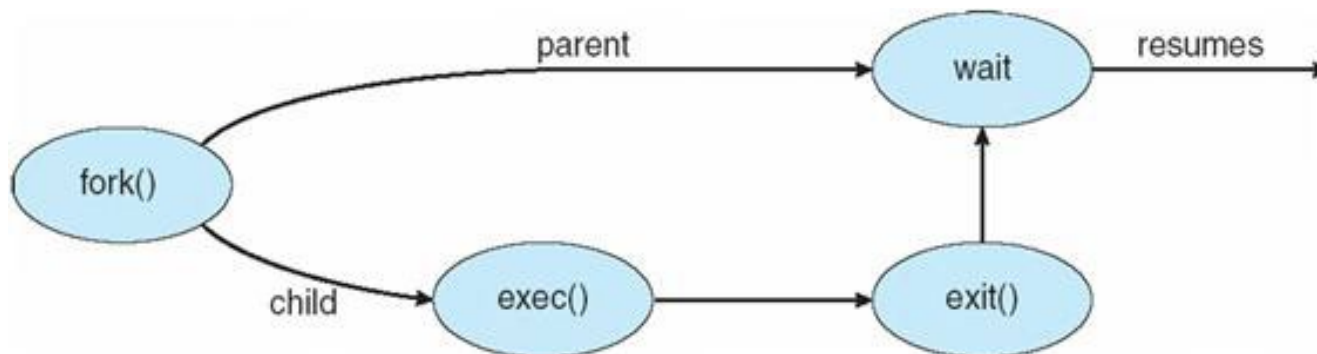
exec()

The exec family of system calls replaces the program executed by a process. When a process calls exec, all code (text) and data in the process is lost and replaced with the executable of the new program.



- Typically, when you want to run a new process, you don't just want another copy of the same program – you want to run a different program. That's what the `exec()` system call is all about.
- It replaces the current running program with an entirely new one.
- This means that when you call `exec`, the operating system stops your process, loads up the new program, and starts that one in its place.
- A process never returns from an `exec()` call (unless there's an error).
- How most programs are run on Unix:

Example code :
check [execl_ls.c](#)



- exec is very useful when you want the child process to run a different program than the parent.
- Take example of the shell program. A shell takes commands as input from the user, runs it and displays the output on screen. Each command corresponds to a new program.
- The number of shell commands in any operating system is huge. It would be very bad code design to append the program or executable associated with all the available commands in a single program.
- In this case, the shell(parent process) can create a child process when a user types a command and let the child process execute the command.

Example code : check [sample.c](#)

Why is `exec()` system call necessary?

- It's not necessary to use `exec()` with `fork()`. If the code that the child will execute is within the program associated with parent, `exec()` is not needed.
- But think of cases when the child has to run multiple programs. Let's take the example of shell program.
- It supports multiple commands like `find`, `mv`, `cp`, `date` etc. Will be it right to include program code associated with these commands in one program or have child load these programs into the memory when required?
- It all depends on your use case.

Environment Variables

- Environment variables are variables that are set up in your shell when you log in.
- They are called “environment variables” because most of them affect the way your Unix shell works for you.
- One points to your home directory and another to your history file. One controls the colors that you see when you ask for a file listing , another sets up your default search path.
- The `env` command will list all of the environment variables and their values.
- Environment variables can be useful when you want to override default settings, or when you need to manage new settings.
- For instance, when you type a command, the only reason your computer knows how to *find* the application corresponding to that command is that the **PATH** environment variable tells it where to look. This variable lists valid directories for your operating system to search for commands, whether that command is `ls` or `cp` for example.

/bin folder

- **/bin: Stands for binary.** This folder contains base executables which are required for minimal system working.
- It's just a directory where a user of an operating system can expect to find applications.
- These commands are available in run-level for basic administration.
- Commands which are available in /bin folder is accessed by every one and can run by every user.
- This folder contains basic commands such as ls, cd, cat, mv, echo and zipping tools such as bzip, gzip etc.



In the `exec*()` functions, the names contain:

- l — list format arguments
- v — vector format arguments
- p — do PATH lookup on the program
- e — take a vector of environment variables too

These combine to give:

- `execl()`
- `execv()`
- `execvp()`
- `execle()`
- `execve()`

Member name	Using pathname	Using filename	Argument List	Argument Array	Original ENV	Provided ENV
exec1()	YES		YES		YES	
exec1p()		YES	YES		YES	
exec1e()	YES		YES			YES
execv()	YES			YES	YES	
execvp()		YES		YES	YES	
execve()	YES			YES		YES
Alphabet used in name		P	I	V		E

Pathname VS Filename

/home/tywong/os/example.c

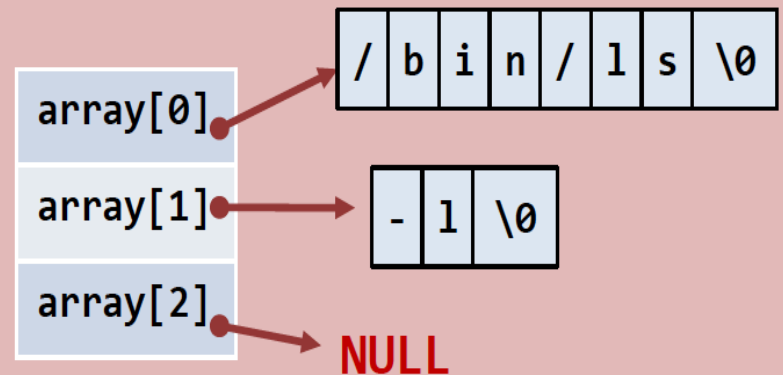
The entire string is a pathname.

The last part is a filename.

Argument list VS Argument array

```
execv("/bin/ls", array);
```

```
execl("/bin/ls",  
"/bin/ls", "-l", NULL);
```



The Command: `"/bin/ls -l"`